SANZI: Local Deployment of Small Language Models on an Apple M1

Muhammad Hasan Al Banna
Technische Universität Darmstadt
Matriculation No.: 2951869
hasan.al@stud.tu-darmstadt.de

Abstract

Most language models are accessed through online services, which means the user's input is processed on infrastructure operated by a third party. This is convenient, but it also reduces the user's direct control over how private or sensitive data is handled. Privacy risks related to large language models have therefore become an important research topic [13]. Local inference provides an alternative in which the model runs on the user's own machine and no external inference service is required.

This paper presents SANZI, a local chatbot for running Llama-3.2-1B-Instruct and Qwen1.5-0.5B-Chat on a MacBook Air with an Apple M1 chip and 8 GB unified memory. The model files are stored locally, prompts are processed by a local Python backend and text generation is performed directly on the device using PyTorch, Hugging Face Transformers [8; 12] and MPS/Metal acceleration. The browser interface communicates only with the local backend, so normal conversations do not depend on an external inference API or third-party server. SANZI supports configurable generation settings, conversation management, token information and FP16, 8-bit Metal and 4-bit Metal inference. The project focuses on keeping local inference practical on limited hardware through context control, model memory management and runtime safeguards. It demonstrates how small language models can be deployed locally on Apple Silicon and how model size, precision, conversation length and memory limitations affect the design of a usable local LLM application.

1 Introduction

Cloud-based language model services normally hide most of the technical details from the user. Model storage, hardware allocation and inference are handled remotely, so using such a service is usually as simple as sending a prompt through a website or an API. Once inference is moved to a personal computer, these hidden limitations become part of the application itself. Memory usage, generation speed, conversation length and numerical precision all have to be considered directly. Local inference also gives the user more control over the data being processed because prompts do not need to be sent to infrastructure operated by an external inference provider [13].

Data location alone does not completely determine who may be legally required to disclose stored information. Article 48 GDPR addresses requests for personal-data disclosure arising from authorities in third countries while the EDPB and EDPS have specifically examined the implications of the U.S. CLOUD Act for the European data-protection framework [5; 6]. Keeping normal inference and conversation data on the user's own device reduces dependence on external providers and the cross-border processing of those prompts.

Smaller instruction-tuned models such as Llama-3.2-1B-Instruct and Qwen1.5-0.5B-Chat make this kind of local deployment possible on consumer hardware [7; 10]. This project studies the problem on a MacBook Air with an Apple M1 chip and 8 GB unified memory. The M1 provides GPU acceleration through Apple's Metal framework and uses a shared memory pool for the CPU and GPU. This makes local neural-network inference possible without a dedicated graphics card, but the available memory is still limited. Model size and precision therefore become important, while longer conversations increase the amount of context that has to be processed during generation.

SANZI was developed to examine how these limitations affect a complete local chatbot rather than only testing whether a model can be loaded. The system runs Llama-3.2-1B-Instruct and Qwen1.5-0.5B-Chat locally using PyTorch and Hugging Face Transformers. It provides a browser-based interface and a separate command-line interface with model switching, configurable

generation settings, conversation management, token information and multiple precision modes. Runtime safeguards are also included for context growth, long generation and memory-related failures. The main objective is to investigate what is required to make small language-model inference practical and usable on limited personal hardware.

This work also examines how model size, precision, memory limits and hardware acceleration affect local LLM deployment.

2 Technical Background

2.1 Small Language Models and Local Inference

The hardware requirements of local language-model inference depend strongly on model size. A model with more parameters normally needs more memory to store its weights and more computation during generation. Smaller language models therefore make local inference possible on consumer devices where memory and processing power are limited. In this project, Llama-3.2-1B-Instruct has about one billion parameters [4] while Qwen1.5-0.5B-Chat is based on a smaller 0.5-billion-parameter model [1].

Both models follow the autoregressive Transformer approach [11]. A user message is first converted into tokens and these tokens are processed by the model as context. The model predicts the next token based on this context, adds it to the sequence and repeats the process until the response is complete. Text generation therefore consists of many consecutive inference steps rather than a single calculation.

Conversation history also affects local inference. Previous user messages and model responses can be included in the input so that the model can follow the ongoing conversation. As the conversation becomes longer, the number of input tokens increases. This means that local inference is affected not only by the size of the model but also by the amount of context that must be processed during generation.

2.2 Hardware for Local Inference

Running a language model locally depends not only on the model itself but also on the hardware available for inference. The processor performs the numerical operations needed during generation while the system memory must hold the model weights and the temporary data created during computation. A GPU can usually process these operations more efficiently in parallel than a CPU, which makes hardware acceleration useful for neural-network inference.

The Apple M1 used for this project combines the CPU and integrated GPU in a single system-on-chip. It also uses unified memory, meaning that both processing units can access the same memory pool. This differs from many systems with a dedicated GPU where system memory and GPU memory are physically separate. For local inference, unified memory can simplify data access but the available memory is shared with macOS and other running applications.

PyTorch supports GPU acceleration on Apple Silicon through the Metal Performance Shaders (MPS) backend [9]. On other computers, similar acceleration can be provided through CUDA when an NVIDIA GPU is available while CPU execution remains the general fallback. These different hardware environments can affect memory availability and inference speed, which makes the choice of execution device an important part of running language models locally.

2.3 Numerical Precision and Quantization

Language-model weights can be stored and processed using different numerical formats. A common option for neural-network inference is FP16, where each value is represented using 16 bits. Compared with higher-precision formats, FP16 reduces memory use and can also improve inference efficiency on supported hardware.

Quantization reduces the numerical precision further. In an 8-bit representation, model weights require roughly half the raw storage of FP16 while a 4-bit representation can reduce the weight storage even more [2; 3]. The actual memory saving is smaller than this simple ratio because other parts of the inference process also consume memory.

Lower precision can make larger models easier to run on devices with limited memory but it also introduces trade-offs. Memory use, loading time, inference speed, hardware support and model behaviour can all change depending on the selected precision. For this reason, FP16, 8-bit and 4-bit configurations provide useful alternatives when evaluating local language-model inference.

3 System Design and Architecture

SANZI separates user interaction from model execution so that the same local language models can be accessed through either a command-line interface or a graphical interface. All model inference remains on the local computer.

3.1 System Components and Interfaces

SANZI provides two ways to interact with the system. The command-line interface allows direct interaction from a terminal while the graphical interface provides a browser-based environment built with Vue 3 and Vite. The GUI communicates with a local Python backend through HTTP requests using JSON data. Both the frontend and backend run on the same computer, so user prompts do not need to be sent to an external inference service.

The backend is organised around two main components. `ChatService` manages the current conversation, selected model and generation settings. `LocalLLMEngine` is responsible for loading the tokenizer and model and for performing inference with PyTorch and Hugging Face Transformers. The command-line interface is implemented separately in `chat_terminal.py` and contains its own model and tokenizer inference path. It therefore does not depend on the graphical interface backend, `ChatService` or `LocalLLMEngine`.

The system supports Llama-3.2-1B-Instruct and Qwen1.5-0.5B-Chat. Only one model is kept active at a time. When the active model or precision configuration is changed, the previous model can be unloaded before the new configuration is loaded. This keeps the runtime structure simple and is particularly useful when working with limited local memory.

3.2 Request and Inference Flow

For the graphical interface, a user message is first sent from the browser to the local backend. The backend receives the request and combines the new message with the conversation state and the selected generation settings. The prepared input is then passed to `LocalLLMEngine`, where the tokenizer converts the conversation into the representation required by the selected model.

The model generates the response locally on the available execution device. On the development MacBook Air M1, inference is performed using Apple MPS when supported. After generation, the output tokens are converted back into text and returned through the backend to the graphical interface. The response is also added to the conversation state so that it can be used as context in later turns.

The CLI follows a separate application path implemented in `chat_terminal.py`. User input is received directly in the terminal and is processed through the CLI's own tokenizer and model inference logic before the generated text is returned to the user. Although the CLI and GUI use different application paths, both rely on PyTorch and Hugging Face Transformers and perform language-model inference locally.

4 Models Used

4.1 Llama-3.2-1B-Instruct

Llama-3.2-1B-Instruct is an instruction-tuned language model from Meta's Llama 3.2 family with approximately one billion parameters [7]. Its relatively small size makes it suitable for studying local inference on consumer hardware where memory and computational resources are limited.

The instruction-tuned version is designed for assistant-like and conversational applications [7]. In SANZI, it serves as the larger of the two selected models and provides a useful reference point for examining how model size affects local deployment requirements and inference behaviour.

4.2 Qwen1.5-0.5B-Chat

Qwen1.5-0.5B-Chat is a compact chat-oriented language model from the Qwen1.5 family [10]. It belongs to the smallest model size released in the Qwen1.5 series, which makes it suitable for local inference where memory and computational resources are limited.

In SANZI, Qwen1.5-0.5B-Chat serves as the lighter of the two selected models. This provides a useful comparison with Llama-3.2-1B-Instruct under the same hardware environment and helps examine how model size affects local deployment requirements and generation performance.

4.3 Model Selection Rationale

The two models were selected to provide a practical comparison between different model sizes while keeping both suitable for local execution. Llama-3.2-1B-Instruct represents the larger option while Qwen1.5-0.5B-Chat provides a smaller and lighter alternative [7; 10].

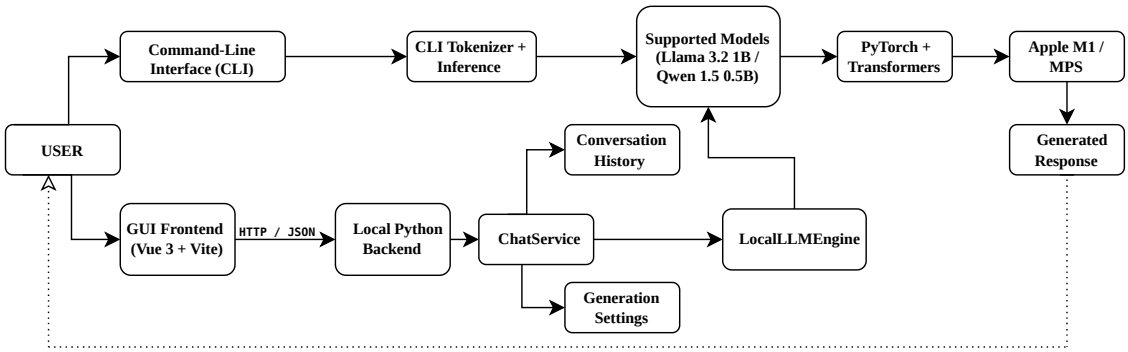


Figure 1: High-level architecture of SANZI showing the separate CLI and GUI application paths and their shared local model execution environment.

Using both models on the same system makes it possible to compare their behaviour under similar hardware conditions. This is useful for observing differences in local deployment requirements, loading time and generation performance without changing the overall application environment. The pair therefore provides a simple basis for studying how model size affects local language-model inference.

5 Implementation

5.1 Command-Line Interface

The command-line interface is the first interaction layer developed for SANZI and runs directly on the local computer. It allows the user to choose between Llama-3.2-1B-Instruct and Qwen1.5-0.5B-Chat before starting a conversation. The selected model is then loaded locally through PyTorch and Hugging Face Transformers without relying on an external inference service or deployment framework.

Several generation parameters can be changed from the CLI. These include the maximum output length, random seed, temperature and top-p value. This gives the user direct control over response length and generation behaviour without changing the source code.

The CLI also provides conversation controls required by the project. The complete conversation can be cleared while a selected number of earlier conversation rounds can also be removed from the beginning of the history. The chatbot session can be terminated directly by the user when it is no longer needed. These controls make the terminal interface useful not only for chatting with the models but also for testing different model and generation configurations during local inference.

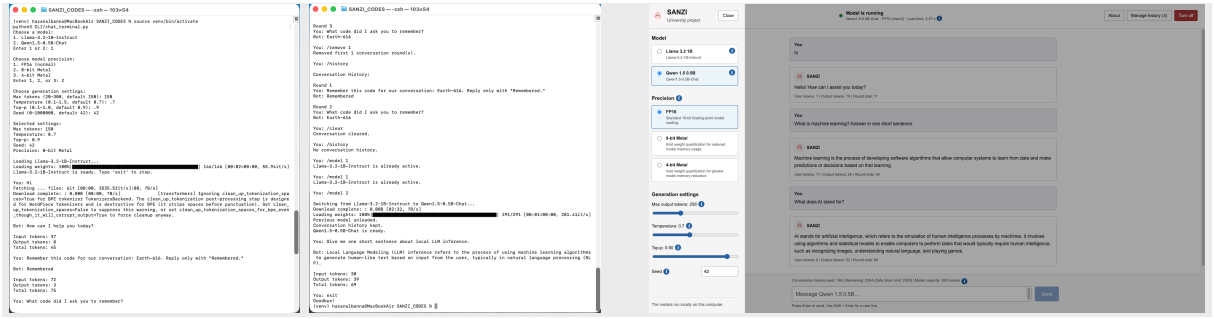
5.2 Graphical User Interface

The graphical user interface provides a browser-based way to use SANZI without working directly in the terminal. The frontend is built with Vue 3 and Vite. Vue 3 was chosen because its component-based and reactive structure makes it convenient to update elements such as chat messages, model selection and generation controls when the application state changes. Vite provides a lightweight development environment with fast startup and simple frontend building, which suited the relatively small scope of SANZI.

The frontend communicates with the local Python backend through HTTP requests using JSON data. This keeps the user interface separate from the model-processing code and provides a simple way to exchange prompts, settings and generated responses. Both the frontend and backend run on the same computer, so this communication remains local and does not require an external inference API.

The GUI exposes the main controls needed for local model interaction. The user can select the active model, enter prompts, view generated responses and adjust settings such as maximum output length, seed, temperature and top-p. Conversation controls are also available through the interface.

The GUI uses the same locally stored Llama and Qwen model families as the CLI but follows its own application path through the local Python backend. This provides a more accessible interface while keeping model execution and conversation data on the user's own machine.



(a) CLI setup (b) CLI management (c) GUI

Figure 2: SANZI user interfaces. (a) CLI setup showing model selection, precision selection and configurable generation settings. (b) CLI management showing conversation history control, round removal, model switching and chatbot termination. (c) GUI showing model and precision selection, generation controls, token monitoring and local chat interaction.

5.3 Conversation Management

Conversation management allows SANZI to maintain context across multiple turns instead of treating every prompt as an independent request. Previous user messages and model responses are stored as conversation history and can be included when preparing the next input. This helps the model respond consistently to follow-up questions and makes the chatbot behave more naturally during longer interactions.

The user can clear the complete conversation when a fresh context is needed. SANZI also allows the first n conversation rounds to be removed from the beginning of the history. This is useful because older messages may no longer be relevant and keeping every previous turn would continuously increase the amount of context that has to be processed.

These controls also support memory management on limited hardware. By reducing unnecessary conversation history, the system can limit the number of input tokens passed to the model and reduce the risk of excessively large prompts during long chat sessions.

5.4 Token and Prompt Handling

Before generation, SANZI converts the current conversation into tokens using the tokenizer of the selected model. The number of tokens is important because both the existing conversation context and the newly generated output consume model context and computational resources. The prepared input is therefore checked before generation.

Both the CLI and GUI use a safe input limit of 2500 tokens. This provides a common safeguard against excessively large conversation contexts during local inference. When the history becomes too long, the user can reduce it using the available conversation controls before continuing the session.

The maximum output length can also be configured but the available ranges differ between the two interfaces. In the CLI, the maximum output setting ranges from 20 to 300 tokens with a default value of 150. In the GUI, the current configuration allows up to 768 output tokens with a default value of 256. This setting limits the number of new tokens that may be generated for a response while the 2500-token safeguard applies to the prepared input context.

6 Hardware and Optimization

6.1 Device Selection and Execution

SANZI was developed and tested on a MacBook Air with an Apple M1 chip and 8 GB of unified memory. The main execution environment therefore uses the PyTorch MPS backend so that model inference can run on the integrated Apple GPU instead of relying only on the CPU [9].

For FP16 execution, the runtime first selects MPS when it is available. The code also contains a CUDA path for systems with an NVIDIA GPU and a CPU fallback when neither GPU backend is available. These alternative FP16 paths are present in the implementation but were not experimentally evaluated in this project. The reported results therefore refer to the Apple M1 and MPS environment.

The 8-bit and 4-bit configurations are different because SANZI implements them using Metal-

based weight quantization. These modes require Apple Silicon with MPS and are therefore part of the Apple-specific execution path used in this project.

6.2 Precision and Quantization

SANZI supports three model precision configurations: FP16, 8-bit Metal and 4-bit Metal. FP16 is used as the standard half-precision execution mode while the lower-bit options are included to reduce model memory usage on Apple Silicon.

The 8-bit and 4-bit modes use Metal-based weight quantization in the current implementation. Reducing the number of bits used for model weights can substantially reduce model memory requirements, as demonstrated by previous work on 8-bit and 4-bit language-model quantization [2; 3]. In practice, the total runtime reduction is not perfectly proportional because activations, tokenizer data, temporary tensors and other components still consume memory.

The lower-bit modes are therefore intended mainly as memory-saving alternatives rather than guaranteed performance improvements. Their effect has to be considered together with loading time, generation speed and model behaviour. The 8-bit and 4-bit implementations are specific to the Apple Metal/MPS path and are not directly portable to CUDA or other non-Apple execution environments. This limitation is discussed further in the reproducibility and limitations sections.

6.3 Memory Management and OOM Handling

Avoiding out-of-memory errors is an important requirement for SANZI because the development machine has only 8 GB of unified memory shared by the operating system and the model runtime. The implementation therefore uses several safeguards rather than relying on a single recovery mechanism.

Only one language model is kept active at a time. When the user changes the model or precision configuration, the previously loaded model is released before the new configuration is loaded. Unused MPS memory can also be cleared during model changes or after a memory-related failure. This helps prevent old model data from remaining allocated unnecessarily.

SANZI also controls the amount of input accepted before generation. The GUI limits an individual user message to 8,000 characters while the prepared conversation is checked against a safe input budget of 2,500 tokens. The character limit prevents unusually large single requests while the token limit controls the actual model context after conversation history and chat formatting are included. Users can remove earlier conversation rounds or clear the complete history when the context becomes too large.

The configurable maximum output length provides another way to control generation workload. If an out-of-memory condition still occurs during model loading or generation, SANZI catches the failure and performs memory cleanup instead of allowing the application to terminate unexpectedly. The user is then advised to reduce the output length, shorten the conversation or use the smaller Qwen model.

Together, the one-active-model policy, explicit memory cleanup, input validation and token safeguards reduce the likelihood of OOM failures and provide a controlled recovery path when memory pressure becomes too high.

7 Experiments and Results

7.1 Experimental Setup

The experiments were performed on the MacBook Air M1 with 8 GB unified memory using the PyTorch MPS backend. Qwen1.5-0.5B-Chat and Llama-3.2-1B-Instruct were each tested with FP16, 8-bit Metal and 4-bit Metal. All runs used seed 42, temperature 0.7, top-p 0.9, a maximum output length of 150 tokens and the same benchmark prompt. Each configuration was measured once, so the results represent practical single-run measurements rather than averaged benchmarks.

7.2 Performance Comparison

Qwen was faster than Llama in all three tested precision modes. Its highest generation speed was obtained with 4-bit Metal at 24.06 tokens/s. For Llama, FP16 was slightly faster than both quantized configurations at 12.42 tokens/s. The quantized Llama models also required longer loading times than FP16.

These results show that lower precision does not automatically improve every performance measure. In this experiment, quantization improved Qwen throughput while Llama showed little generation-speed benefit. All six configurations completed successfully without runtime errors.

Model	Precision	Load (s)	Generation (s)	Output tokens	Tokens/s	Status
Qwen-0.5B	FP16	2.19	6.99	107	15.31	Success
Qwen-0.5B	8-bit Metal	1.68	7.75	150	19.35	Success
Qwen-0.5B	4-bit Metal	1.77	5.86	141	24.06	Success
Llama-1B	FP16	2.56	12.07	150	12.42	Success
Llama-1B	8-bit Metal	4.23	12.76	150	11.75	Success
Llama-1B	4-bit Metal	4.44	12.74	150	11.77	Success

Table 1: Single-run performance measurements on the Apple M1 using MPS.

Requirement	Status
Local CLI chatbot	Verified
User-selectable Llama-3.2-1B-Instruct and Qwen1.5-0.5B-Chat	Verified
Configurable maximum output length, seed, temperature and top-p	Verified
Clear the complete conversation	Verified
Remove the first n conversation rounds	Verified
Turn off the chatbot	Verified
OOM safeguards and recovery	Verified
Local execution without Ollama, llama.cpp or another deployment framework	Verified
Additional graphical user interface	Implemented

Table 2: Validation of the main seminar project requirements.

During informal interaction tests, Llama-3.2-1B-Instruct generally produced more coherent and reliable responses than Qwen1.5-0.5B-Chat. Qwen occasionally produced incorrect or hallucinated answers despite achieving higher generation speed. Since response quality was not evaluated using a formal benchmark, this observation should be treated as qualitative rather than a quantitative comparison.

7.3 Functional Validation

The implemented system was also checked against the main functional requirements of the seminar project. Table 2 summarizes the verified features.

The CLI and GUI demonstrations in Figure 2 show several of these functions in operation. The token safeguards and memory-management mechanisms are described in Section 6.

8 Reproducibility

The SANZI source code is organised into separate components for the command-line interface, graphical interface and local inference backend. The `CLI/` directory contains the terminal chatbot while `backend/` contains the Python model engine and local server. The Vue 3/Vite interface is stored in `frontend/`. Model download scripts for Llama and Qwen are included at the project root along with `benchmark_models.py` and the benchmark results used in Section 7.

A fresh installation can be created using the dependencies listed in `backend/requirements.txt` while the frontend dependencies are defined by `package.json` and `pnpm-lock.yaml`. The model files do not need to be distributed with the source code because `download_llama.py` and `download_qwen.py` reproduce the required local model directories. The `README` documents environment creation and the commands required to run the CLI, backend, frontend and benchmark. The local virtual environment is not required because a new environment can be created on the target machine.

The complete configuration used for the experiments is directly reproducible on compatible Apple Silicon hardware with MPS. The FP16 implementation also contains CUDA and CPU execution paths, allowing the main chatbot functionality to run on non-Apple systems. However, the 8-bit and 4-bit modes use the Apple Metal/MPS quantization path and therefore require a different quantization backend on NVIDIA or other non-Apple hardware. At the final validation stage, 49 automated tests passed, including backend regression tests and focused tests for CLI model-loading OOM recovery.

9 Limitations and Future Work

The main limitation of SANZI is that the 8-bit and 4-bit quantization paths are specific to Apple Metal/MPS. Although FP16 execution includes CUDA and CPU fallbacks, these alternatives were not experimentally evaluated. The benchmark also used only one run per configuration and response quality was assessed informally rather than through a systematic accuracy or hallucination

benchmark.

Future work could extend low-bit quantization to CUDA-based systems and repeat the experiments across multiple runs and hardware platforms. A formal response-quality evaluation and broader automated testing of the graphical interface would also provide a more complete assessment of the system.

10 Conclusion

SANZI demonstrates that small language models can be deployed as a complete local chatbot on consumer hardware without relying on an external inference service. The project implements both a command-line interface and a graphical interface with support for Llama-3.2-1B-Instruct and Qwen1.5-0.5B-Chat. Hardware-aware execution, configurable generation settings, conversation management, quantization and memory safeguards were integrated into the system and the main seminar requirements were successfully verified.

The experiments showed that performance depends on both model choice and numerical precision. Qwen achieved higher generation speed, with the 4-bit Metal configuration reaching the best measured throughput, while Llama appeared more reliable during informal interaction tests. Quantization improved Qwen throughput but did not provide the same benefit for Llama, showing that lower precision does not automatically improve every model. Overall, SANZI shows that practical local LLM deployment depends not only on choosing a small model but also on balancing hardware support, memory limits, numerical precision and model behaviour.

References

[1] Jinze Bai, Shuai Bai, Yunfei Chu, Zeyu Cui, Kai Dang, Xiaodong Deng, Yang Fan, Wenbin Ge, Yu Han, Fei Huang, et al. 2023. Qwen technical report. *arXiv preprint arXiv:2309.16609*.

[2] Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. 2022. Llm.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems*.

[3] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. Qlora: Efficient finetuning of quantized llms. In *Advances in Neural Information Processing Systems*.

[4] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*.

[5] European Data Protection Board. 2025. Guidelines 02/2024 on article 48 gdpr. https://www.edpb.europa.eu/documents/guideline/guidelines-022024-on-article-48-gdpr_en. Final version.

[6] European Data Protection Board and European Data Protection Supervisor. 2019. Joint response to the libe committee on the impact of the us cloud act on the european legal framework for personal data protection. https://www.edpb.europa.eu/documents/edpb-correspondence/edpb-edps-joint-response-to-the-libe-committee-on-the-impact-of-the_en.

[7] Meta. 2024. Llama 3.2 1b instruct. https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct.

[8] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32.

[9] PyTorch. 2026. Mps backend. https://docs.pytorch.org/docs/stable/notes/mps.html. Accessed August 2026.

[10] Qwen Team. 2024. Introducing qwen1.5. https://qwenlm.github.io/blog/qwen1.5/.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.

[12] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Remi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45.

[13] Biwei Yan, Kun Li, Minghui Xu, Yueyan Dong, Yue Zhang, Zhaochun Ren, and Xiuzhen Cheng. 2024. On protecting the data privacy of large language models (llms): A survey. *arXiv preprint arXiv:2403.05156*.